as a side effect, languages that are very similar to English. Also, the simple backend is only capable of reading translations but cannot dynamically store them to any format.
That does not mean you're stuck with these limitations, though. The Ruby I18n gem makes it very easy to exchange the Simple backend implementation with something else that fits better for your needs. E.g. you could exchange it with Globalize's Static backend:

```
I18n.backend = Globalize::Backend::Static.new
```

You can also use the Chain backend to chain multiple backends together. This is useful when you want to use standard translations with a Simple backend but store custom application translations in a database or other backends. For example, you could use the Active Record backend and fall back to the (default) Simple backend:

```
I18n.backend =
I18n::Backend::Chain.new(I18n::Backend::ActiveRecord.new,
I18n.backend)
```

6.2 Using Different Exception Handlers

The I18n API defines the following exceptions that will be raised by backends when the corresponding unexpected conditions occur:

```
MissingTranslationData       # no translation was found for the
requested key
InvalidLocale                # the locale set to I18n.locale is
invalid (e.g. nil)
InvalidPluralizationData     # a count option was passed but the
translation data is not suitable for pluralization
MissingInterpolationArgument # the translation expects an
interpolation argument that has not been passed
ReservedInterpolationKey     # the translation contains a reserved
interpolation variable name (i.e. one of: scope, default)
UnknownFileType              # the backend does not know how to handle
a file type that was added to I18n.load_path
```

The I18n API will catch all of these exceptions when they are thrown in the backend and pass them to the default_exception_handler method. This method will re-raise all exceptions except for `MissingTranslationData` exceptions. When a `MissingTranslationData` exception has been caught, it will return the exception's error message string containing the missing key/scope.
The reason for this is that during development you'd usually want your views to still render even though a translation is missing.

In other contexts you might want to change this behavior, though. E.g. the default exception handling does not allow to catch missing translations during automated tests easily. For this purpose a different exception handler can be specified. The specified exception handler must be a method on the I18n module or a class with `#call` method:

```
module I18n
  class JustRaiseExceptionHandler < ExceptionHandler
    def call(exception, locale, key, options)
      if exception.is_a?(MissingTranslationData)
        raise exception.to_exception
      else
        super
      end
    end
  end
end
```